

第13讲 实验设计

用数值实验观察“局部变化率”

- 不是性能实验，而是概念验证实验。
- 固定问题 -> 预测 -> 扰动 -> 记录 -> 对照理论 -> 解释。

本讲最低目标

学完以后，你不需要会证明极限，但必须能够用自己的话解释：函数是什么、斜率是什么、导数为什么是“局部敏感度”，以及为什么深度学习需要它。

1. 实验目标

通过最小 NumPy
实验，把导数从抽象符号变成可以观察的现象。实验不使用自动求导，避免把底层机制再次隐藏起来。

2. 研究问题（RQ）

- RQ1：直线 $y=3x+2$ 在不同位置的数值导数是否都约等于 3？
- RQ2：曲线 $y=x^2$ 的局部斜率是否随 x 改变，并与 $2x$ 一致？
- RQ3：当 h 逐渐减小时，割线斜率是否逼近 $x=3$ 处的导数 6？
- RQ4：对 $y=w*x+b$ ，数值估计的 dy/dw 是否等于 x ？
- RQ5：ReLU 在负区间和正区间的局部斜率是否分别约为 0 和 1？

3. 假设

H1

所有 RQ 都应理论与理论一致；数值误差主要来自有限差分近似与浮点精度。

4. 自变量 / 因变量 / 控制变量

实验	自变量	观测量	控制
1	x	数值导数	函数固定为 $3x+2$
2	x	数值导数	函数固定为 x^2
3	h	割线斜率误差	$x_0=3$
4	w 扰动	输出变化/数值导数	$x=2.5, b=0.4$
5	x 位置	ReLU数值导数	h 固定

5. 实验步骤

步骤1：先在纸上写出你对每个实验的预测。

步骤2：运行 `lesson13_derivative_experiment.py`。

步骤3：重点检查打印出的数值导数，不要只看图片。

步骤4：比较理论值与数值值；解释误差来自哪里。

步骤5：打开两张 PNG：一张看割线如何逼近切线，一张看 ReLU 的分段斜率。

6. 核心实现

```
def numerical_derivative(f, x, h=1e-5):  
    return (f(x + h) - f(x - h)) / (2 * h)
```

中心差分一次调用 $f(x+h)$ 与

$f(x-h)$ 。如果把这种办法直接用于数十亿参数，就会极其昂贵，因此后面需要 Backpropagation。

7. 预期结果

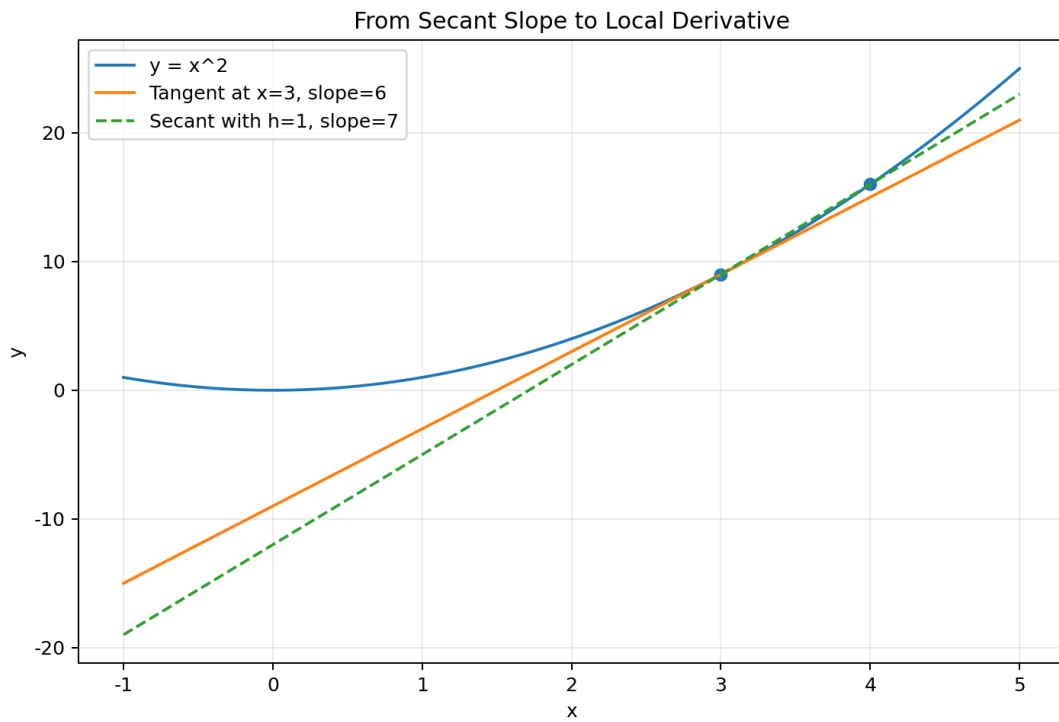
实验1：不同 x 位置都约为 3。

实验2： $x=-2,-1,0,1,3$ 时数值导数分别约为 -4,-2,0,2,6。

实验3： h 从 1 降到 $1e-5$ 时，割线斜率从 7 逐渐逼近 6。

实验4： dy/dw 约为 2.5； w 增加 0.01 时 y 增加约 0.025。

实验5：ReLU 在负数处约为 0，在正数处约为 1； $x=0$ 是特殊折点。



8. 实际结果摘要

本次代码已实际运行。关键结果：

```
linear derivative: 3.000000
```

```
x^2 at x=3: numerical derivative = 6.000000
```

```
y=w*x+b, x=2.5: dy/dw = 2.500000
```

```
ReLU: derivative approximately 0 for x<0, 1 for x>0
```

完整终端输出见“第13讲_实验实际输出.txt”。

9. 如何写实验结论

结论模板

实验结果支持 H1：数值差分能够在简单函数上恢复理论局部斜率。由此可以把导数理解为“当前点附近的局部敏感度”。Weight

实验进一步说明，神经网络参数发生小变化时，导数可以预测输出的一阶变化。这为后续 Loss、Gradient 与 Backpropagation 提供了可观察基础。

10. 常见实验错误

错误 1：只看最终数字

必须解释这个数字在回答哪个 RQ。

错误 2：把 $x=0$ 处 ReLU 的中心差分 0.5 当成严格导数

0.5 是特定数值近似方式得到的结果；ReLU 在 0 处左右斜率不同，严格导数不存在。

错误 3：h 盲目设成 $1e-30$

有限精度计算会导致消减误差。数值实验不是数学上的无限小。

错误 4：把数值导数当训练算法

本实验用于理解和检查；真正网络训练依赖自动微分/反向传播。